

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет ИУ
Кафедра ИУ5

Курс «Парадигмы и конструкции языков программирования»
Отчет по рубежному контролю №2 Вариант
Б34

Выполнил:
Студент группы ИУ5-32Б
Девятов И.А.
Подпись и дата:

Проверил:
Преподаватель каф. ИУ5
Гапанюк Ю.Е.
Подпись и дата:

Москва, 2025

Текст программы

main.py

main.py

class Procedure:

def __init__(self, id, name, size, database_id):

self.id = id

self.name = name

self.size = size

self.database_id = database_id

class Database:

def __init__(self, id, name):

self.id = id

self.name = name

class ProcedureDatabase:

def __init__(self, procedure_id, database_id):

self.procedure_id = procedure_id

self.database_id = database_id

def build_one_to_many(databases, procedures):

return [

(p.name, p.size, d.name)

for d in databases

for p in procedures

if p.database_id == d.id

]

def build_many_to_many(databases, procedures, procedures_databases):

many_to_many_temp = [

(d.name, pd.database_id, pd.procedure_id)

for d in databases

for pd in procedures_databases

if d.id == pd.database_id

]

return [

```

        (p.name, db_name)
    for db_name, db_id, proc_id in many_to_many_temp
    for p in procedures
    if p.id == proc_id
]

```

```

def query_b1(one_to_many):
    return sorted(one_to_many, key=lambda x: x[0])

```

```

def query_b2(one_to_many, databases):
    res = []
    for d in databases:
        count = len([item for item in one_to_many if item[2] == d.name])
        if count > 0:
            res.append((d.name, count))
    res.sort(key=lambda x: x[1])
    return res

```

```

def query_b3(many_to_many):
    return sorted(
        [(name, db) for name, db in many_to_many if name.endswith("Proc")],
        key=lambda x: x[0]
    )

```

```

def main():
    databases = [
        Database(1, "HR_DB"),
        Database(2, "Finance_DB"),
        Database(3, "Analytics_DB"),
        Database(4, "Common_DB"),
    ]

    procedures = [
        Procedure(1, "GetUsers", 120, 1),
        Procedure(2, "CalcSalary", 180, 2),
        Procedure(3, "ReportGen", 90, 3),
        Procedure(4, "LogCleanup", 60, 4),
        Procedure(5, "BackupProc", 150, 4),
    ]

```

```

        Procedure(6, "DataMov", 200, 3),
    ]

    procedures_databases = [
        ProcedureDatabase(1, 1),
        ProcedureDatabase(2, 2),
        ProcedureDatabase(3, 3),
        ProcedureDatabase(4, 4),
        ProcedureDatabase(5, 4),
        ProcedureDatabase(6, 3),
        ProcedureDatabase(3, 4),
        ProcedureDatabase(4, 1),
    ]

    one_to_many = build_one_to_many(databases, procedures)
    many_to_many = build_many_to_many(databases, procedures, procedures_databases)

    print("--- Запрос Б1 ---")
    print("Список связанных процедур и баз данных (1:M), отсортированный по имени процедуры:")
    for proc, size, db in query_b1(one_to_many):
        print(f" Процедура: {proc}, Размер: {size} строк, База: {db}")

    print("\n--- Запрос Б2 ---")
    print("Список баз данных с количеством процедур, отсортированный по количеству:")
    for name, count in query_b2(one_to_many, databases):
        print(f" База данных: {name}, Количество процедур: {count}")

    print("\n--- Запрос Б3 ---")
    print("Список процедур, название которых заканчивается на 'Proc', и базы данных (M:M):")
    for proc, db in query_b3(many_to_many):
        print(f" Процедура: {proc}, База данных: {db}")

if __name__ == "__main__":
    main()
test_main.py
import unittest
from main import (
    Database,
    Procedure,
    ProcedureDatabase,
    build_one_to_many,

```

```
build_many_to_many,  
query_b1,  
query_b2,  
query_b3,  
)
```

```
class TestProcedures(unittest.TestCase):
```

```
    def setUp(self):
```

```
        self.databases = [  
            Database(1, "HR_DB"),  
            Database(2, "Finance_DB"),  
            Database(3, "Analytics_DB"),  
            Database(4, "Common_DB"),  
        ]
```

```
        self.procedures = [  
            Procedure(1, "GetUsers", 120, 1),  
            Procedure(2, "CalcSalary", 180, 2),  
            Procedure(3, "ReportGen", 90, 3),  
            Procedure(4, "LogCleanup", 60, 4),  
            Procedure(5, "BackupProc", 150, 4),  
            Procedure(6, "DataMov", 200, 3),  
        ]
```

```
        self.procedures_databases = [  
            ProcedureDatabase(1, 1),  
            ProcedureDatabase(2, 2),  
            ProcedureDatabase(3, 3),  
            ProcedureDatabase(4, 4),  
            ProcedureDatabase(5, 4),  
            ProcedureDatabase(6, 3),  
            ProcedureDatabase(3, 4),  
            ProcedureDatabase(4, 1),  
        ]
```

```
    def test_build_one_to_many(self):
```

```
        """Тест построения отношения один-ко-многим"""
```

```
        result = build_one_to_many(self.databases, self.procedures)
```

```
        self.assertEqual(len(result), 6) # 6 процедур, каждая привязана к одной базе
```

```
        self.assertTrue(all(isinstance(item, tuple) for item in result))
```

```
        for name, size, db in result:
```

```
            self.assertIsInstance(name, str)
```

```

        self.assertIsInstance(size, int)
        self.assertIsInstance(db, str)

def test_query_b1_sorted_by_name(self):
    """Тест запроса Б1: сортировка по имени процедуры"""
    one_to_many = build_one_to_many(self.databases, self.procedures)
    result = query_b1(one_to_many)

    names = [item[0] for item in result]
    self.assertEqual(names, sorted(names))

    self.assertEqual(len(result), 6)

def test_query_b3_filter_by_proc_suffix(self):
    """Тест запроса Б3: фильтрация по окончанию 'Proc'"""
    many_to_many = build_many_to_many(
        self.databases, self.procedures, self.procedures_databases
    )
    result = query_b3(many_to_many)

    self.assertTrue(all(name.endswith("Proc") for name, _ in result))

    self.assertEqual(len(result), 1)
    self.assertEqual(result[0][0], "BackupProc")

    self.assertIn(result[0][1], ["Analytics_DB", "Common_DB"])

if __name__ == "__main__":
    unittest.main()

```

Скриншот работы программы

```
● (venv) deviatovilya@MacBook-Air-cua-Deviatov rk2 % python -m unittest test_main.py
...
-----
Ran 3 tests in 0.000s

OK
● (venv) deviatovilya@MacBook-Air-cua-Deviatov rk2 % python test_main.py
...
-----
Ran 3 tests in 0.000s

OK
● (venv) deviatovilya@MacBook-Air-cua-Deviatov rk2 % python main.py
--- Запрос Б1 ---
Список связанных процедур и баз данных (1:M), отсортированный по имени процедуры:
Процедура: BackupProc, Размер: 150 строк, База: Common_DB
Процедура: CalcSalary, Размер: 180 строк, База: Finance_DB
Процедура: DataMov, Размер: 200 строк, База: Analytics_DB
Процедура: GetUsers, Размер: 120 строк, База: HR_DB
Процедура: LogCleanup, Размер: 60 строк, База: Common_DB
Процедура: ReportGen, Размер: 90 строк, База: Analytics_DB

--- Запрос Б2 ---
Список баз данных с количеством процедур, отсортированный по количеству:
База данных: HR_DB, Количество процедур: 1
База данных: Finance_DB, Количество процедур: 1
База данных: Analytics_DB, Количество процедур: 2
База данных: Common_DB, Количество процедур: 2

--- Запрос Б3 ---
Список процедур, название которых заканчивается на 'Proc', и базы данных (M:M):
Процедура: BackupProc, База данных: Common_DB
```

Рис. 1 Результат работы программы